

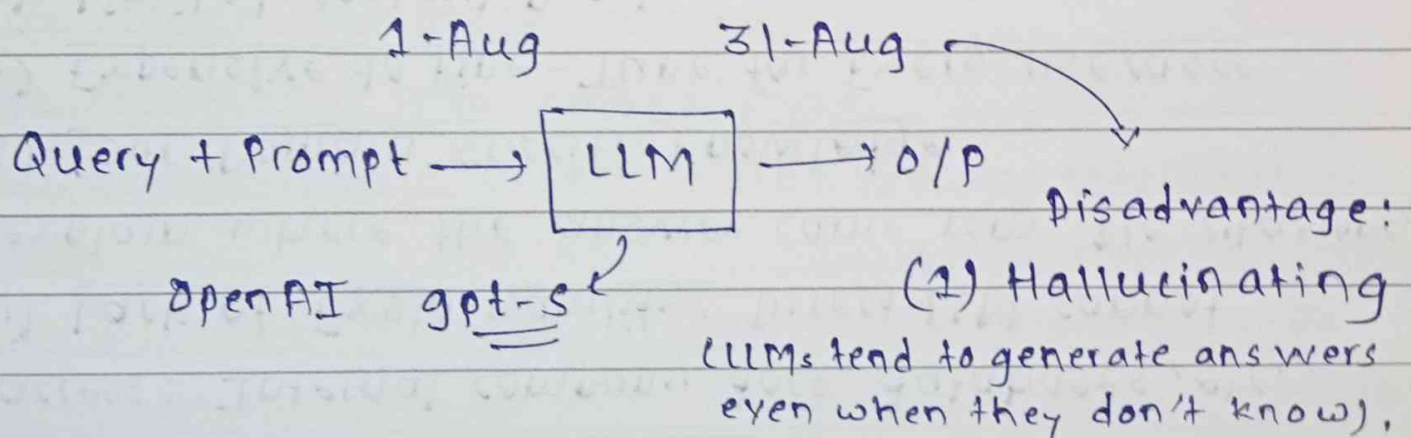
Retrieval Augmented Generation (RAG)

* The process of optimizing the output of a large language model, so that it references an authoritative knowledge base outside of its training data sources before generating a response.

* LLMs are trained on vast volumes of data and use billions of parameters to generate original output.

* RAG extends the already powerful capabilities of LLMs to specific domains or an organization's internal knowledge base, all without the need to retrain the model.

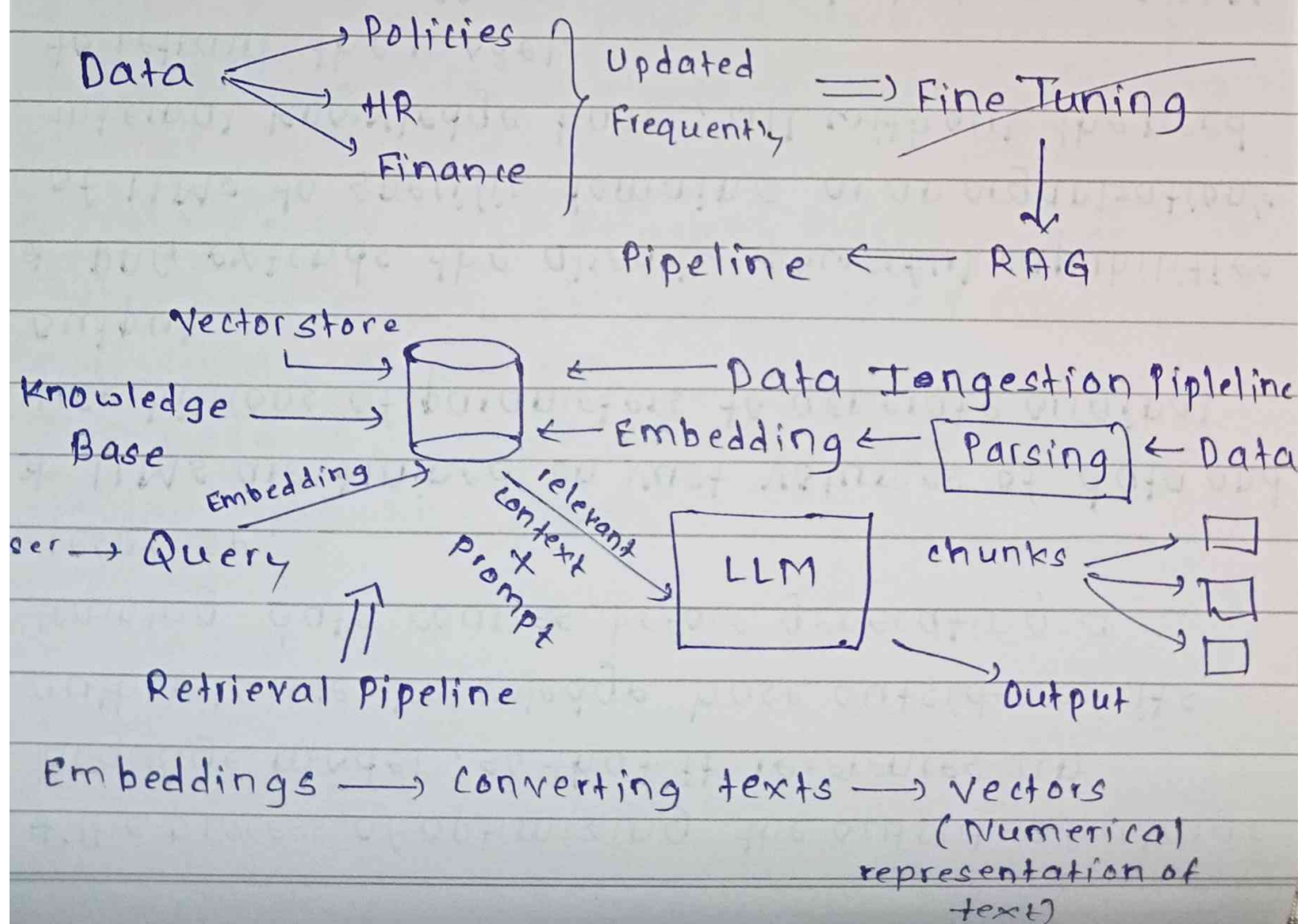
* It is a cost-effective approach to improve LLM output.

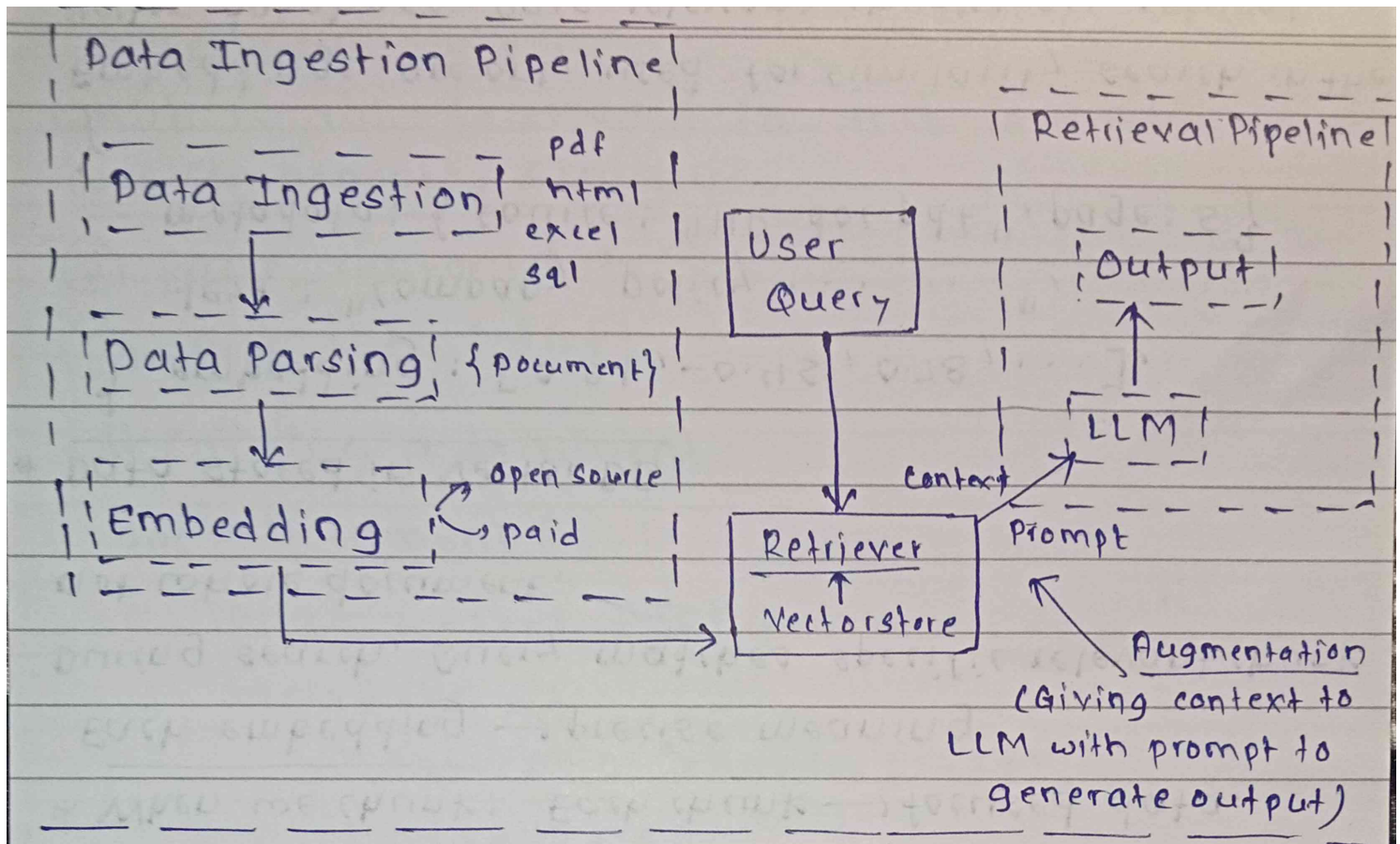


RAG reduces hallucination by grounding answers in retrieved documents.

(Hallucination - Incorrect but confident answer)

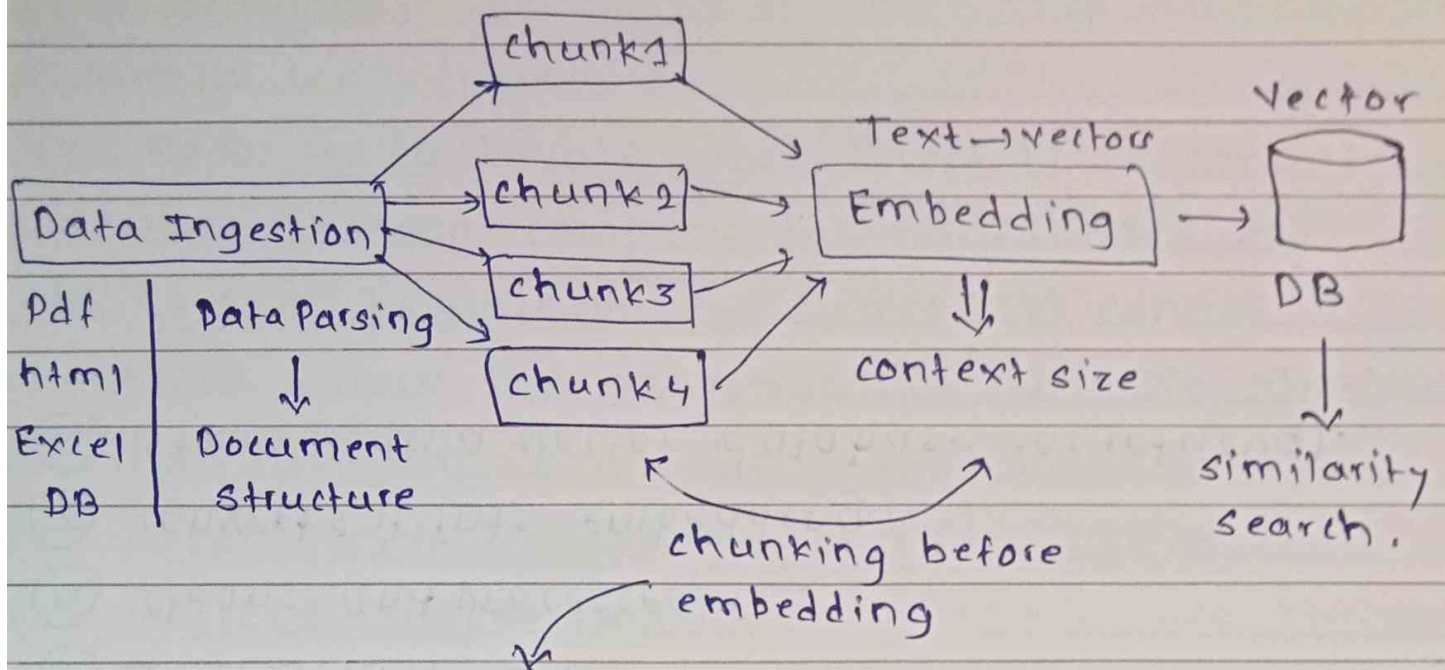
- (2) Outdated Knowledge: A direct LLM only knows what is was trained on.
- (3) No Access to Private Data: Direct LLMs cannot access: Internal company docs, databases, etc.
- (4) Lack of Explainability: Direct LLM cannot explain where the answer came from. No citations.
- (5) Poor Domain Specific Knowledge
- (6) Expensive to Fine-Tune for Every usecases
- (7) Limited context Passing





* Data Ingestion: Preparing knowledge so that LLM can use it intelligently later.

- ① collects raw data.
- ② cleans and processes it.
- ③ converts it into embeddings.
- ④ stores it in a vector database for retrieval.



* Avoiding Context Dilution: If we embed the whole document, important info gets 'washed out' & model can't distinguish fine details.

* When we chunk: Each chunk $\rightarrow$ focused data.

Each embedding $\rightarrow$ precise meaning

During search, Query matches specific relevant chunk not whole document.

Data stored in Vector DB:

```
{
  embedding : [0.21, -0.45, 0.78, ...],
  text : "Company policy . . . . .",
  metadata : { source : "HR-doc.pdf", page : 5 }
}
```

Embeddings are only used for similarity search in the vector database. Once relevant chunks are retrieved, their original text is passed to LLM as context.

* Langchain Document → A kind of data structure which is able to save data in a format.

Core components / structure:

① page_content (str)

② metadata (dict)

creating a document:

```
doc = Document (
```

```
    page_content = "RAG is a . . . . .",
```

```
    metadata = {
```

```
        "source" : "chapter1.pdf",
```

```
        "page" : 5,
```

```
    }    "timestamp" : "2024-01-05"
```

* Data Loading → Loaders convert raw data into Documents.

E.x. PDF → multiple Documents (one per Page).

* chunking → A large document becomes multiple smaller documents.

* Embedding → Each document converted to embedding.

* Langchain Document Loader: Loads data from external sources & converts it into Langchain document object.

It is the first step in Data Ingestion Process.

E.x. PDFLoader, CSV Loader, WebBaseLoader.

```
from langchain.document_loadersimport import PyPDFLoader
loader = PyPDFLoader("file.pdf")
documents = loader.load()
```

* Directory Loader: Load multiple files from a folder / directory and convert them into Langchain Document objects automatically.

```
from langchain.document_loaders import
DirectoryLoader
loader = DirectoryLoader("data/");
documents = loader.load()
```

Now documents contains all files as structured Documents.

* Chunking:

(1) sentence-transformer - Python library available on hugging face, used to convert sentences or text into dense vector embeddings.

```
from sentence_transformers import SentenceTransformer  
model = SentenceTransformer('all-MiniLM-L6-V2')  
embedding = model.encode(" - - - ");
```

Embedding size ← A pretrained embedding model from
= 384 dimensions the Sentence-Transformers ecosystem.

all-MiniLM-L6-V2 ← Version
↑
Trained on ← Lightweight ← Six transformer
diverse data transformer model Layer

Some other Sentence-Transformer model:

① multi-qa-MiniLM-L6-cos-v1

Optimized for Q & A retrieval.

* Other Embedding Options:

* API-Based models - text-embedding-3-small
(OpenAI embedding)

* Hugging Face Transformers - BERT, RoBERTa

* Embedding: A function that maps text $\rightarrow$ a vector in high-dimensional space.

* How it is created:

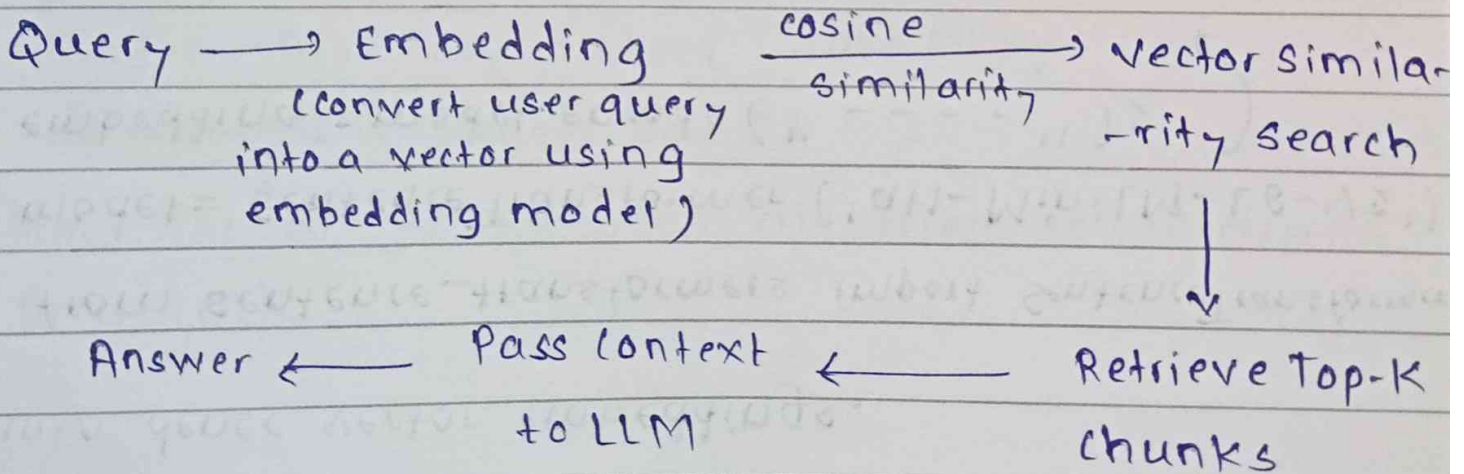
- (1) Converts words $\rightarrow$ numbers (Tokenization).
- (2) Pass through neural network (transformer layers).
- (3) Extract a fixed size output.

* Compare of two embedding using cosine similarity

$$\cos \theta = \frac{A \cdot B}{|A| |B|}$$

Higher Score $\rightarrow$ more similar meaning
Lower Score $\rightarrow$ less relevant.

* Retrieval System: Finds the most relevant information (chunks) for a user's query from a knowledge base.



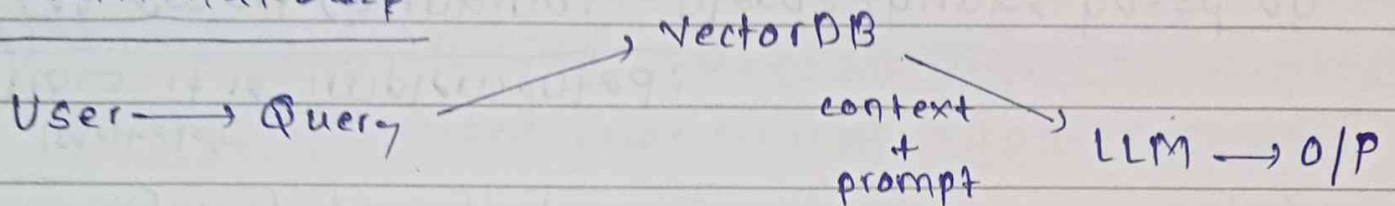
* Agentic RAG:

2 Important components;

RAG + Autonomous Agent

↓
Intelligent, dynamic
decisions about the

Traditional RAG:



Autonomous Agent know:

- (1) When to retrieve info,
- (2) what to retrieve,
- (3) where to retrieve
- (4) How many times to retrieve,

* Multi-Tenant Architecture: One system serves multiple users/organizations (tenants), while keeping their data isolated.

Tenant: A separate user or organization using our system.

User A → Doc A → Embeddings A → Vector DB (A partition)

User B → Doc B → Embeddings B → Vector DB (B partition)

* Stateful scanning of PDF in RAG:

Stateful scanning of a PDF means processing the document while remembering context across pages / chunks — instead of treating every chunk independently.

In a basic RAG pipeline: Each chunk is processed independently (stateless)

Issues: (i) Broken context across pages.

(ii) Tables split incorrectly.

* Stateful —> The system keeps track of what it has already seen.

while scanning a PDF, it remembers:

* Previous headings.

* Section structure

* Ongoing paragraphs

* Table continuity.

(Methods)

* How it is implemented:

(1) Smart chunking: Merge chunks based on paragraph continuity / token limits.

(2) Sliding window:

chunk 1: tokens 0 - 200

chunk 2: tokens 150 - 350

Overlap preserves context.

Thread Pool Executor:

* Parallel Processing: Running multiple task at the same time instead of one-by-one

ThreadPoolExecutor is a Python class used to run tasks in parallel using threads.

Working:

1) Thread Pool: A fixed no. of threads (max_workers)
Threads are reused (not created again & again)

2) Task Submission: Two main ways:

① map():

executor.map(function, iterable)

Simple, returns results in order.

② submit():

future = executor.submit(task, s)

print(future.result())

* Returns a future object → A result that will be available later.

* Semantic chunking: Splitting text into chunks based on meaning, not just size.

* Fixed-size chunking:
chunk every 200 words.

* Semantic chunking:

Split when topic/meaning changes.

* How semantic chunking works:

(1) Sentence splitting: Break text into sentences.

(2) Convert sentences $\rightarrow$ embeddings

Using models like Sentence Transformers.

(3) Measure similarity $\rightarrow$ compare consecutive sentences:

* If similarity is high $\rightarrow$ same chunk

* If similarity is low $\rightarrow$ new chunk

(4) Group semantically similar sentences.